

SkillForge: Self-Distilling Agents for Project-Specific Issue Resolution

Silin Chen*, Han Li*, Xiaodong Gu†, Yuling Shi, Haibing Guan

Shanghai Jiao Tong University

cslsolow@gmail.com, lihan0421@sjtu.edu.cn, xiaodong.gu@sjtu.edu.cn

yuling.shi@sjtu.edu.cn, hbguan@sjtu.edu.cn

Abstract—Large language model (LLM) based agents have demonstrated remarkable proficiency in automated software issue resolution, yet they often struggle to resolve issues in a specific repository because they lack project-specific knowledge. Existing self-evolving approaches acquire such knowledge from repository history or online repair trajectories, but they either depend on available historical issue-resolution signals or incur substantial per-issue test-time exploration cost. In this paper, we propose SkillForge, a self-distillation framework that proactively acquires project-specific knowledge from the repository itself. Instead of waiting for real issues to expose project-specific knowledge gaps, SkillForge synthesizes project-specific issues by re-implementing test-covered core functionalities of the repository. By resolving these synthetic issues, SkillForge distills reusable project-specific knowledge into entity-grounded skills and associates them with relevant repository entities for future issue resolution. Extensive experiments using both open-source and closed-source models show that SkillForge consistently improves issue resolution performance over strong baselines. These results demonstrate that proactively acquiring project-specific knowledge before solving real issues substantially improves downstream software issue resolution¹.

Index Terms—Software engineering agents, software issue resolution, large language models

I. INTRODUCTION

Large language model (LLM) based agents have demonstrated remarkable capabilities in software issue resolution [1]–[15]. By autonomously navigating repositories, invoking tools, and iteratively proposing patches, modern software engineering (SWE) agents—such as SWE-agent [16] and OpenHands [17]—can resolve complex, real-world bugs that previously required expert human intervention. Driven by increasingly powerful foundation models and better scaffolding designs, these agents have achieved impressive results on standard benchmarks such as SWE-bench [3], [18]–[23], demonstrating strong generalization across diverse codebases and issue types.

Despite these advances, a critical bottleneck emerges when agents are deployed on a specific project: they typically must solve issues from scratch, without project-specific knowledge. Real-world projects often exhibit structural regularities: recurring failures may involve the same brittle modules, correct

patches may need to preserve repository-specific API contracts, and related APIs are often coupled through implicit execution paths. Agents that lack such project-specific knowledge must repeatedly rediscover these conventions during issue resolution. This limitation creates a cold-start problem: before project-specific knowledge has been acquired, even a highly capable agent is reduced to a generic repository explorer and can repeatedly fall into the same project-specific pitfalls.

To address this knowledge gap, recent works have explored self-evolving SWE agents that acquire project-specific knowledge from prior issue-resolution signals [2], [24]–[29]. History-driven methods such as EvoCoder [24] and SWE-Exp [2] distill reusable knowledge from historical issues, commits, or agent trajectories. Online methods instead refine the agent during test-time exploration by generating additional trajectories on the current issue. While effective, both paradigms acquire project-specific knowledge reactively. History-driven methods are bounded by the richness and coverage of past issues: repository behaviors, API combinations, and long-tail components that have not been exercised by historical trajectories provide little learning signal. Online methods reduce this dependence on history, but they pay substantial trajectory, token, and time costs for each target issue [30], and the acquired guidance becomes available only after the issue has already arrived.

In this paper, we propose SkillForge, a proactive self-distillation framework for addressing the cold-start problem in project-specific issue resolution. Rather than waiting for real issue-resolution history to accumulate, SkillForge synthesizes project-specific issues from the current repository. Given a repository snapshot, SkillForge starts from test-covered core functionalities, follows execution traces to identify the code regions that jointly implement the same functionality, and rewrites these coordinated segments under constrained context. The resulting synthetic issues are executable and behaviorally grounded by the repository tests. A SWE agent then resolves these synthetic issues, producing trajectories from which SkillForge distills project-specific knowledge.

SkillForge organizes the distilled knowledge as a dual-level skill repository. The global diagnostic skill set ($\mathcal{M}_{ext}$) captures reusable project-level knowledge for diagnosis and navigation, including entity roles, reasoning playbooks, and related APIs. The local intervention skill set ($\mathcal{M}_{int}$) records entity-specific

*Silin Chen and Han Li contributed equally to this work.

†Corresponding author: Xiaodong Gu.

¹Our code and data are available at <https://github.com/cslsolow/SkillForge>

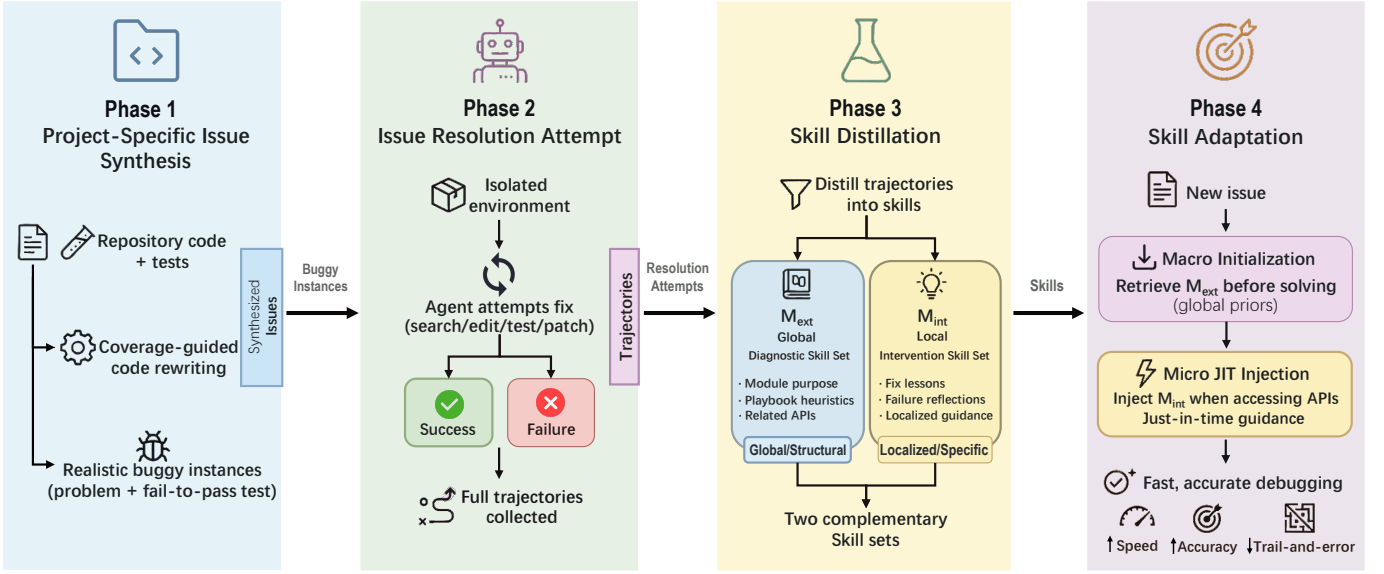


Fig. 1: Overview of SkillForge.

modification guidance and pitfall-avoidance lessons distilled from successful and failed resolution trajectories. During downstream issue resolution, SkillForge first retrieves relevant global diagnostic skills as initial project-specific context, and then injects local intervention skills just in time whenever the agent accesses corresponding repository entities. This design treats skills as one concrete representation of project-specific knowledge, while keeping retrieval aligned with the agent’s current code interaction context.

We implement SkillForge with Mini-SWE-Agent and evaluate it on SWE-bench Verified [31] and SWE-bench Pro [32] using both DeepSeek-V3.2 and GPT-5-mini. On SWE-bench Verified, SkillForge achieves 72.2% and 60.6% Pass@1, improving over Mini-SWE-Agent by +5.8 and +5.6 percentage points, respectively. On SWE-bench Pro, SkillForge further improves over Mini-SWE-Agent by +5.8 and +4.1 percentage points. Across both benchmarks, SkillForge outperforms all evaluated history-driven and online project-specific knowledge acquisition baselines available for each benchmark, showing that proactive project-specific knowledge acquisition can improve issue resolution without relying on rich historical issue trajectories or heavy per-issue online exploration.

The main contributions of this paper are summarized as follows:

- We propose a novel self-distillation paradigm for addressing the cold-start problem in project-specific issue resolution. SkillForge proactively acquires project-specific knowledge from a repository’s own tests and code, without relying on rich historical issue-resolution trajectories or costly per-issue online exploration.
- We design a dual-level skill repository that represents project-specific knowledge as global diagnostic skills and local intervention skills, enabling entity-grounded retrieval and just-in-time guidance during downstream

issue resolution.

- Empirically, SkillForge improves over Mini-SWE-Agent by +5.8%/+5.6% on SWE-bench Verified and +5.8%/+4.1% on SWE-bench Pro for DeepSeek-V3.2 and GPT-5-mini, respectively, outperforming all evaluated history-driven and online project-specific knowledge acquisition baselines available for each benchmark.

II. METHODOLOGY

SkillForge is a self-distillation framework for addressing the cold-start problem in project-specific issue resolution. The key idea is to proactively derive project-specific knowledge directly from the repository itself, rather than waiting for real-world issue-resolution trajectories to accumulate. To achieve this, SkillForge synthesizes project-specific issues, resolves them with a SWE agent, and distills the resulting project-specific knowledge into reusable skills that can be retrieved during subsequent real-world issue resolution.

Figure 1 presents an overview of SkillForge. Given a repository snapshot and its test suite, the framework consists of four stages: (1) generating project-specific synthetic issues by rewriting core functionalities; (2) resolving these issues with a SWE agent through iterative patch generation, tool execution, and test feedback; (3) distilling project-specific knowledge from the resulting resolution trajectories and organizing it as reusable skills in a dual-level skill repository, comprising a global diagnostic skill set that captures reusable project-level knowledge and a local intervention skill set that records entity-specific guidance; and (4) retrieving the relevant skills during downstream real-world issue resolution whenever the corresponding repository entities are encountered. Each stage is described in detail below.

A. Project-Specific Issue Synthesis

To acquire project-specific knowledge without relying on historical issue reports, SkillForge synthesizes project-specific issues by rewriting functionality-critical code segments and treating the resulting test failures as resolution targets. Rather than explicitly injecting handcrafted bugs, the framework asks an LLM to reimplement repository functionality under constrained context, encouraging the rewritten code to naturally exhibit implementation mistakes similar to those introduced during real software development. More importantly, because the model must reconstruct functionality without access to the original implementation, the rewritten code reflects its general coding knowledge rather than the repository’s project-specific knowledge, thereby exposing the project-specific behaviors that the agent needs to learn. Throughout this process, supervision is derived solely from the repository’s codebase and test suite, without relying on historical issue reports or human-written bug descriptions.

Test-driven scope and trace. Given a code repository $\mathcal{R}$, SkillForge identifies test cases that exercise core repository functionalities, since these behaviors provide the most informative supervision for project-specific knowledge acquisition [33], [34]. For each passed test case, SkillForge executes it under coverage instrumentation to obtain an execution trace: the set of source files and line ranges that are exercised. This trace defines the candidate code regions for rewriting since modifications within executed regions are more likely to affect observable test outcomes. We then extract contiguous code segments from the traced regions—either complete functions or cohesive execution fragments, and attach surrounding context (e.g., preceding and following lines) for each segment.

Select critical segments. The number of traced segments can be large. We use an LLM to select a small set of *critical* segments that are most likely to represent the functionality of this test case. The LLM is given the test’s purpose and scenario (derived from the test code or description) along with summaries of the candidate segments (file, line range, and code content). It returns a ranked subset of top- k segments with brief justifications. This step keeps the subsequent rewriting phase focused on high-impact regions. Unlike function-level rewriting approaches, SkillForge may select multiple critical segments exercised by the same test case, enabling synthetic issues that capture cross-component interactions within the repository.

Code rewriting. For each selected segment, we ask an LLM to *rewrite* the corresponding code by re-completing the local functionality. Crucially, the LLM is not provided with the original implementation; instead, it receives only limited contextual information: the lines before and after the segment, the segment’s location and indentation, and a high-level description of the test’s goal. The model is prompted to produce a plausible alternative implementation that preserves the intended API while potentially simplifying logic. Rather than explicitly injecting predefined faults, this “strict-mask” design exposes the gap between an agent’s general coding

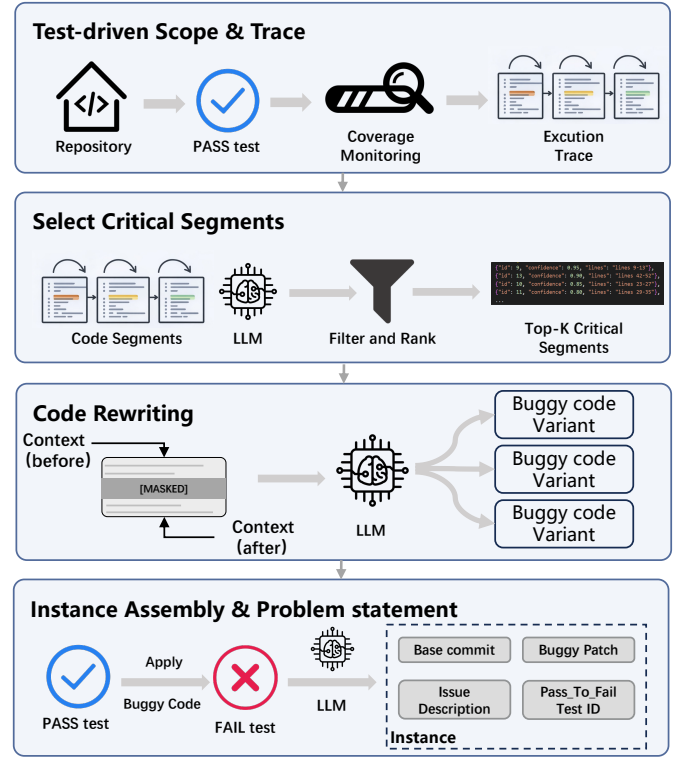


Fig. 2: Synthesize project-specific issues.

knowledge and the repository’s project-specific knowledge by inducing realistic implementation mistakes under constrained context.

Instance assembly and problem statement. Once the rewritten segments induce test failures, we construct a buggy repository snapshot and derive both a buggy patch and a reference patch. Following prior synthetic SWE task construction works [35]–[37], we execute the failing tests and use an LLM to convert the resulting failure evidence into a user-facing problem statement without exposing implementation details or repair hints. The resulting synthetic instance consists of a base commit, a buggy patch, a reference patch, a problem statement, and the associated test cases, matching the standard SWE-bench issue-resolution format. These synthetic instances serve as self-supervised tasks for acquiring project-specific knowledge before real issue resolution.

Issue Resolution Attempt For each synthesized instance, SkillForge follows the SWE-bench-style issue resolution protocol: the agent is placed in an isolated repository environment, receives the problem statement and buggy codebase, and attempts to produce a patch that resolves the failing tests. We record the resulting action trajectory (e.g., file edits, shell commands, and test runs) for subsequent skill distillation.

B. Skill Distillation

The objective of this stage is to distill project-specific knowledge from synthetic issue-resolution trajectories and organize it as reusable skills. Formally, given a trajectory $\mathcal{T} = \{(a_1, o_1), \dots, (a_n, o_n)\}$ where a_i and o_i denote the agent’s action and the environment’s observation at step i , our

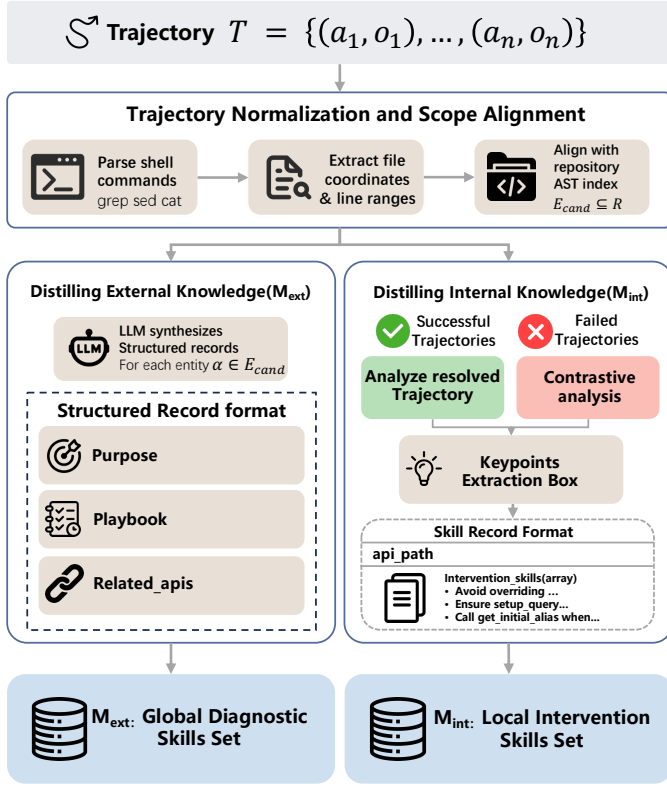


Fig. 3: Skill distillation.

extraction pipeline transforms $\mathcal{T}$ into two complementary skill sets within the dual-level skill repository: $\mathcal{M}_{ext}$ and $\mathcal{M}_{int}$.

Trajectory Normalization and Entity Alignment. Before skill distillation, we normalize the trajectories and align them with the repository’s entities. Given a target repository $\mathcal{R}$, we perform a structured analysis of the trajectory $\mathcal{T}$ to identify code-access events. By parsing shell commands (e.g., `grep`, `sed`, `cat`) in the trajectory, we extract a set of accessed files, represented by coordinates and line ranges. We then align these coordinates with the repository’s structural index—an AST-derived mapping from source files to their class and function hierarchies—to resolve them into cohesive code scopes (e.g., frequently accessed files or core modules). Beyond trajectory-derived accesses, files surfaced in the agent’s submission diff, the golden patch, and the test patch are additionally incorporated to ensure coverage of ground-truth edit sites. The resulting collection is denoted as a set of candidate entities $\mathcal{E}_{cand} \subseteq \mathcal{R}$. This step ensures that the extracted skills are strictly grounded in the actual codebase structure, preventing the LLM from hallucinating non-existent interfaces during subsequent extraction.

Distilling Global Diagnostic Skills ($\mathcal{M}_{ext}$). $\mathcal{M}_{ext}$ represents project-specific diagnostic knowledge as global diagnostic skills distilled from synthetic issue-resolution trajectories. Unlike repository summaries that describe what a module contains, $\mathcal{M}_{ext}$ captures how an agent should reason about a repository entity during issue resolution, including where to begin debugging, which APIs are jointly involved,

and what repository-specific behaviors should be considered. These skills are organized as structured skill records and associated with the corresponding repository entities for future retrieval. For each candidate entity $\alpha \in \mathcal{E}_{cand}$, an LLM analyzes the corresponding resolution trajectories together with the repository context to distill three complementary aspects of project-specific knowledge: (i) the entity’s functional role in issue resolution, (ii) reusable reasoning strategies repeatedly validated during issue resolution, and (iii) project-specific API interactions that emerge during problem solving. These are represented as the `purpose`, `playbook`, and `related_apis` fields, respectively. For example,

```
{
  "api_path": "django/db/models/sql/compiler.py",
  "purpose": "Compiles Django ORM queries into SQL
              statements, ...",
  "playbook": "If the issue involves SQL ..., first
              check if setup_query properly ...",
  "related_apis": [
    { "api_path": "django/db/models/sql/query.py",
      "reason": "Provides the Query class that
                  manages alias ..." },
    ... ]
}
```

The design of each key is specifically tailored to enhance the agent’s autonomous capabilities: (1) The `purpose` field identifies the role an entity plays during issue resolution, helping the agent determine whether it is a relevant debugging entry point rather than merely summarizing its implementation. (2) The `playbook` field captures project-specific reasoning patterns distilled from trajectories. Rather than generic debugging advice, it records repository-specific knowledge that has been validated through interaction with the repository. (3) The `related_apis` field records APIs that are repeatedly co-involved during issue resolution. Unlike static dependency graphs, these relationships reflect repository-specific interaction patterns observed during problem solving, allowing the agent to navigate cross-module behaviors that are not evident from structural dependencies alone.

Unlike repository summarization, which captures static code semantics, our objective is to distill actionable project-specific knowledge that emerges only when an agent attempts to solve project-specific issues.

Distilling Local Intervention Skills ($\mathcal{M}_{int}$). In contrast, $\mathcal{M}_{int}$ stores project-specific intervention knowledge that guides how a repository entity should be modified during issue resolution. Unlike $\mathcal{M}_{ext}$, which supports repository navigation and diagnosis, $\mathcal{M}_{int}$ captures concrete repair knowledge distilled from issue-resolution trajectories. This knowledge is represented as local intervention skills associated with individual repository entities. We distill this intervention knowledge from both successful and failed resolution trajectories. Successful trajectories reveal repair strategies that consistently lead to correct implementations, while failed trajectories expose repository-specific pitfalls by contrasting incorrect patches with the corresponding reference patches. The resulting knowledge is transformed into actionable intervention skills and indexed by repository entities. Combining successful and

failed trajectories provides complementary supervision: the former reinforces effective repair patterns, whereas the latter helps the agent avoid recurring mistakes when modifying the same repository entities in future issue-resolution tasks. For example,

```
{
  "api_path": "django/db/models/sql/compiler.py",
  "intervention_skills": [
    "Avoid overriding ... because ...",
    ...]
}
```

Conceptually, $\mathcal{M}_{ext}$ captures diagnostic skills for understanding repository entities and planning repository navigation, whereas $\mathcal{M}_{int}$ captures intervention skills for modifying those entities based on project-specific knowledge distilled from synthetic issue resolution.

C. Skill Adaptation

To utilize the acquired project-specific knowledge during downstream issue resolution, SkillForge retrieves the distilled skills associated with relevant repository entities and injects them into the agent’s reasoning process. This retrieval follows a two-stage, context-aware mechanism consisting of macro-level initialization and micro-level just-in-time (JIT) intervention.

Macro-level Initialization with Global Diagnostic Skills.

Before the agent begins resolving a new issue, we first establish a global semantic context using the diagnostic knowledge represented by the global skill set ($\mathcal{M}_{ext}$). Given the new issue description as a query, we employ a BM25 retriever to identify the top- k most relevant skill records from $\mathcal{M}_{ext}$. The selected records—comprising the API paths, their purposes, and the associated playbooks—are prepended to the agent’s initial prompt as project-specific priors. This equips the agent with an immediate, high-level understanding of the repository’s architecture and relevant previously distilled heuristics.

Micro-level JIT Injection of Local Intervention Skills.

While the macro-level injection provides initial guidance, flooding the context with all local intervention skills ($\mathcal{M}_{int}$) at once would introduce significant noise. Instead, SkillForge dynamically injects $\mathcal{M}_{int}$ based on the agent’s real-time actions. At each interaction step, SkillForge monitors the agent’s executed shell commands to extract the specific file paths it is currently accessing. If an accessed file matches an entry in $\mathcal{M}_{int}$, the corresponding intervention cues and reflections are appended to the agent’s context as an auxiliary observation for the subsequent reasoning step. This dynamic mechanism ensures that the agent receives targeted, pitfall-avoidance guidance exactly when it navigates to a relevant codebase region, ensuring the injected skills are strictly aligned with the agent’s current code interaction context.

Unlike existing methods that retrieve semantically similar records from a centralized knowledge store, SkillForge grounds project-specific knowledge directly to repository entities. Consequently, retrieval is triggered by the agent’s interaction with the corresponding code entities rather than solely by semantic similarity. This entity-grounded design ensures

that diagnostic and intervention knowledge is delivered precisely when the agent reaches the relevant repository context, reducing retrieval ambiguity while maintaining tight alignment between the injected knowledge and the code under inspection.

III. EXPERIMENTAL SETUP

A. Research Questions

We aim to evaluate SkillForge by answering four research questions (RQs):

RQ1 (Effectiveness of SkillForge): What is the impact of SkillForge on issue resolution performance compared to other self-evolving methods?

RQ2 (Ablation Study): How do the different components of SkillForge individually affect its effectiveness?

RQ3 (Impact of Hyperparameters): How do the key hyperparameters influence the overall performance of SkillForge?

RQ4 (Efficacy across Diverse Repositories): Does SkillForge consistently demonstrate effectiveness across various repositories?

B. Datasets

We evaluate SkillForge on **SWE-bench Verified** [31], a human-validated benchmark released by OpenAI, which has been widely adopted as the standard for evaluating software engineering agents. The benchmark consists of 500 tasks sourced from popular Python repositories on GitHub.

We additionally evaluate on the full **SWE-bench Pro** [32], consisting of 731 instances across Python, JavaScript, TypeScript, and Go repositories. This benchmark contains more challenging long-horizon software engineering tasks that better reflect realistic multi-file issue resolution.

C. Baseline Methods

We compare SkillForge with representative approaches that acquire project-specific knowledge from different supervision sources.

History-driven project-specific knowledge acquisition. These methods acquire project-specific knowledge from historical issue-resolution issues, and reuse the distilled knowledge to assist future issue resolution.

- **SWE-Exp** [2]: A framework that distills diagnostic patterns and repair strategies from prior agent trajectories, utilizing a dual-agent architecture to provide actionable guidance for new tasks.
- **EvoCoder** [24]: A multi-agent continual learning framework that uses trajectory-based reflection to progressively refine problem-solving strategies based on previously resolved cases.
- **MemGovern** [25]: MemGovern converts human debugging traces from GitHub into structured memory, enabling code agents to leverage prior problem-solving knowledge for improved issue resolution.

Online project-specific knowledge acquisition. These approaches adapt agents during the resolution of the current issue by distilling project-specific knowledge from substantial online reasoning trajectories through test-time exploration:

- **SAGE** [27]: A plan-learning method that distills reusable problem-solving guidance from ongoing agent attempts to iteratively improve performance on the current instance.
- **SWE-Debate** [28]: A multi-agent competitive framework that evolves stronger fix strategies through structured debate over competing reasoning trajectories.
- **Live-SWE-agent** [29]: A live self-evolving software agent that autonomously updates its own scaffold during runtime while solving software engineering tasks.

Variants of SkillForge. To isolate the source of project-specific knowledge in SkillForge, we compare against two controlled variants while keeping the same downstream reused mechanism:

- **SkillForge w/ SWE-Smith.** A controlled synthesis variant that replaces our functionality-level repository probing with SWE-Smith single-function rewriting.
- **SkillForge w/ LLM Summary.** A static repository-understanding variant that replaces trajectory-distilled project-specific knowledge with LLM-generated repository summaries.

D. Metrics

We evaluate SkillForge from both effectiveness and cost-efficiency perspectives.

Pass@1 denotes the percentage of issues successfully resolved on the first attempt, following the evaluation protocol of [1], [16]. It directly measures the framework’s ability to generate correct patches without relying on repeated repair iterations.

Avg Cost represents the average end-to-end monetary cost per evaluated issue, measured in U.S. dollars [30]. The reported cost amortizes offline pre-computation over the evaluated instances and includes all stages of the framework pipeline, including synthetic issue generation, trajectory collection, skill distillation, and online issue resolution. MemGovern [25] constructs a pre-constructed $\sim 150\text{K}$ -card experience base from public GitHub issues using GPT-5.1; since the associated preprocessing cost is unavailable, it is excluded from Avg Cost.

E. Implementation Details

We implement SkillForge with Mini-SWE-Agent [38], a bash-based agent scaffold [27], [39], [40], and evaluate DeepSeek-V3.2 [41] and GPT-5-mini [42] with default inference settings.

For SWE-bench Verified, synthesis is temporally isolated. For each target instance, we first roll back the repository before both the golden patch and golden test patch; on this rolled-back snapshot, an LLM localizes semantically relevant tests because extracting the full test suite from every snapshot is time- and compute-intensive. We synthesize issues only from this filtered subset, yielding 577 synthesized issues. SkillForge itself does not require historical issue-resolution data; in the temporal evaluation protocol, the agent may retrieve only skills distilled from synthesized issues in the same repository whose source

TABLE I: Main results on SWE-bench Verified.

Method	Model	Pass@1	Avg Cost
Mini-SWE-Agent	DeepSeek-V3.2	66.4%	\$0.049
	GPT-5-mini	55.0%	\$0.031
<i>History-driven project-specific knowledge acquisition</i>			
SWE-Exp	DeepSeek-V3.2	69.0% [†] +2.6%	\$0.090
	GPT-5-mini	56.6% [†] +1.6%	\$0.065
EvoCoder	DeepSeek-V3.2	67.0% +0.6%	\$0.064
	GPT-5-mini	58.4% +3.4%	\$0.052
MemGovern	DeepSeek-V3.2	69.2% [†] +2.8%	–
	GPT-5-mini	58.0% [†] +3.0%	–
<i>Online project-specific knowledge acquisition</i>			
SAGE	DeepSeek-V3.2	67.2% +0.8%	\$0.081
	GPT-5-mini	56.0% [†] +1.0%	\$0.052
SWE-Debate	DeepSeek-V3.2	68.2% +1.8%	\$0.382
	GPT-5-mini	56.4% +1.4%	\$0.167
Live-SWE-agent	DeepSeek-V3.2	67.0% +0.6%	\$0.050
	GPT-5-mini	55.6% +0.6%	\$0.042
<i>Variants of SkillForge</i>			
SkillForge w/ SWE-Smith	DeepSeek-V3.2	68.0% +1.6%	\$0.088
	GPT-5-mini	56.4% [†] +1.4%	\$0.071
SkillForge w/ LLM Summary	DeepSeek-V3.2	68.7% [†] +2.3%	\$0.069
	GPT-5-mini	54.4% +0.6%	\$0.065
SkillForge	DeepSeek-V3.2	72.2% [†] +5.8%	\$0.074
	GPT-5-mini	60.6% [†] +5.6%	\$0.066

[†]: $p - \text{value} < 0.05$.

commits predate the target instance’s gold commit, ensuring that no current-instance or future information is used.

For the two variants, we extract relevant functions from the baseline agent’s trajectory, then apply either SWE-Smith-style single-function rewriting or an LLM summary of how those functions are jointly used; both keep SkillForge’s skill format and injection interface. Across all stages, temperature is 0, the action budget is 250 steps, BM25 retrieves top-5 skills, and Table I reports Pass@1 averaged over three runs.

IV. RESULTS

A. RQ1: Effectiveness of SkillForge

Table I summarizes the main results on SWE-bench Verified. SkillForge achieves 72.2% and 60.6% Pass@1 with DeepSeek-V3.2 and GPT-5-mini, outperforming all baselines by absolute margins of **+5.8%** and **+5.6%** over Mini-SWE-Agent. Among history-driven project-specific knowledge acquisition methods, MemGovern is the strongest competitor (69.2%/58.0%), yet SkillForge still surpasses it by +3.0%/+2.6%. One possible reason is that history-driven project-specific knowledge acquisition methods are bounded by the coverage and quality of repository history: knowledge distilled from prior repository histories only reflects previously observed issue types, often remains coarse or instance-tied, and external debugging discussions improve coverage only by weakening repository specificity. Among online project-specific knowledge acquisition methods, SkillForge also surpasses SAGE (67.2%/56.0%) and SWE-Debate (68.2%/56.4%), with gains of +5.0%/+4.6% and +4.0%/+4.2%, respectively, while avoiding the high per-issue exploration cost of methods such as SWE-Debate. For DeepSeek-V3.2, it also outperforms Live-SWE-agent (67.0%,

TABLE II: Main results on SWE-bench Pro.

Method	DeepSeek-V3.2		GPT-5-mini	
	Pass@1	Avg Cost	Pass@1	Avg Cost
Mini-SWE-Agent	28.3%	\$0.047	47.6%	\$0.063
SWE-Exp	29.4% $\uparrow 1.1\%$	\$0.083	48.7% $\uparrow 0.9\%$	\$0.089
Live-SWE-agent	32.4% $\uparrow 4.1\%$	\$0.051	49.1% $\uparrow 1.5\%$	\$0.072
SkillForge	34.1% $\uparrow 5.8\%$	\$0.069	51.7% $\uparrow 4.1\%$	\$0.087

$\dagger$: p -value < 0.05 .

\$0.050) by +5.2%. Compared with Mini-SWE-Agent and history-driven project-specific knowledge acquisition methods, the main cost difference of SkillForge lies in the offline pre-computation stage, where it synthesizes project-specific issues and distills skills from their resolution trajectories. Therefore, the reported Avg Cost includes amortized offline pre-computation; once this repository-level skill repository is built, the online cost of resolving each real issue remains close to that of the underlying agent.

Table II further reports results on SWE-bench Pro. SkillForge achieves 34.1% Pass@1 with DeepSeek-V3.2 and 51.7% with GPT-5-mini, improving over Mini-SWE-Agent by +5.8% and +4.1%, respectively, with both gains statistically significant (p -value < 0.05). It also outperforms the strongest available Pro baselines, exceeding Live-SWE-agent by +1.7%/+2.6% and SWE-Exp by +4.7%/+3.2% under the two backbones.

Among the controlled variants, SkillForge w/ SWE-Smith reaches 68.0%/\$0.088 and 56.4%/\$0.071 under DeepSeek-V3.2 and GPT-5-mini, respectively, while SkillForge w/ LLM Summary reaches 68.7%/\$0.069 and 54.4%/\$0.065. These results show that skill injection alone is insufficient: both simply summarizing trajectory-extracted functions and rewriting one function at a time underperform the full method. SWE-Smith rewrites one function at a time, whereas SkillForge starts from a repository functionality exposed by a core test, follows its execution trace, and rewrites multiple coordinated functions or code segments that are naturally composed to realize that functionality. This functionality-level synthesis captures inter-function coordination, API coupling, and project-specific repair pitfalls, while more naturally surfacing the current LLM’s repository-misaligned coding tendencies.

Finding 1

SkillForge outperforms all evaluated history-driven and online project-specific knowledge acquisition baselines available for each benchmark, improving over Mini-SWE-Agent by **+5.8%/+5.6%** on SWE-bench Verified and **+5.8%/+4.1%** on SWE-bench Pro.

B. RQ2: Ablation Study

We investigate two questions: (1) whether both forms of project-specific knowledge are necessary for issue resolution, and (2) whether project-specific knowledge distilled for one LLM backbone transfers to another.

a) Component ablation.: Table III reports the effect of removing each type of project-specific knowledge. Removing

TABLE III: Ablation study results.

Approach	DeepSeek-V3.2	GPT-5-mini
w/o Global Diagnostic Skills	68.4% ($\downarrow 3.8\%$)	57.6% ($\downarrow 3.0\%$)
w/o Local Intervention Skills	67.8% ($\downarrow 4.4\%$)	57.2% ($\downarrow 3.4\%$)
SkillForge	72.2%	60.6%

TABLE IV: Cross-LLM skill transfer on SWE-bench Verified. Resolver LLM denotes the backbone used for real issue resolution, while Knowledge-source LLM denotes the backbone used to synthesize instances and distill skills.

Resolver LLM	Knowledge-source LLM	Pass@1
GPT-5-mini	GPT-5-mini	60.6%
	DeepSeek-V3.2	55.0%
DeepSeek-V3.2	GPT-5-mini	65.2%
	DeepSeek-V3.2	72.2%

$\mathcal{M}_{ext}$ leads to drops of 3.8% and 3.0%, demonstrating the importance of repository-level diagnostic knowledge for issue resolution. Removing $\mathcal{M}_{int}$ causes slightly larger drops of 4.4% and 3.4%, showing that project-specific intervention knowledge distilled from synthetic issue-resolution trajectories is equally critical. These results indicate that effective use of project-specific knowledge requires both repository-level diagnostic knowledge and entity-level intervention knowledge, which provide complementary guidance throughout the issue-resolution process.

b) Transferability of project-specific knowledge across LLM backbones.: We further investigate whether the project-specific knowledge distilled by SkillForge is transferable across LLM backbones. Specifically, we use one LLM to synthesize project-specific issues, resolve them, and distill the resulting project-specific knowledge, which is represented as reusable skills. These skills are then injected during issue resolution performed by either the same or a different LLM. Table IV exhibits a clear diagonal pattern: each resolver LLM achieves the best performance when paired with project-specific knowledge distilled from itself. DeepSeek-V3.2 reaches 72.2% when using its own distilled skills but drops to 65.2% when using GPT-5-mini’s. Similarly, GPT-5-mini achieves 60.6% with its own distilled skills but decreases to 55.0% when using those distilled by DeepSeek-V3.2. These results indicate that the distilled project-specific knowledge is not universally transferable across LLMs. Instead, it captures how a particular LLM reasons about and edits a specific repository. Since different LLMs exhibit different coding priors, the repository-specific mismatches exposed during synthetic issue resolution are likewise model dependent, leading to different project-specific knowledge being distilled. Consequently, knowledge distilled from one LLM provides limited guidance for another and may even introduce irrelevant context.

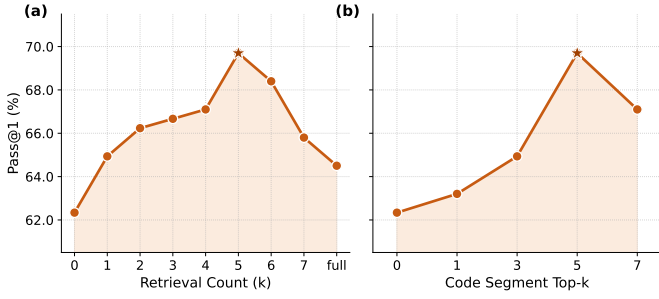


Fig. 4: Hyperparameter study results on the Sphinx and Django repositories with GPT-5-mini. (a) BM25 skill retrieval count. (b) Code segment rewritten count during issue synthesis. Red stars mark the best-performing k in each setting.

Finding 2

Effective issue resolution requires both repository-level diagnostic knowledge and entity-level intervention knowledge. Moreover, the distilled project-specific knowledge is LLM-specific rather than universally transferable across backbones.

C. RQ3: Impact of Hyperparameters

We investigate the sensitivity of SkillForge to two key hyperparameters governing project-specific knowledge acquisition and utilization: (1) the number of retrieved global diagnostic skill records per issue (k_r), and (2) the number of code segments rewritten to synthesize project-specific issues (k_s).

To evaluate k_r , we conducted a hyperparameter study on the Django and Sphinx repositories using GPT-5-mini, varying k_r from 0 to 7, as well as a “full” condition where all relevant skill records were retrieved. Figure 4(a) reports Pass@1 as a function of k_r across the two repositories. Without retrieving project-specific knowledge ($k_r = 0$), the agent achieves 62.3%. Performance improves steadily as k_r increases, reaching its peak at $k_r = 5$ (69.7%). Beyond this point, the full retrieval condition drops slightly to 67.5%, suggesting that lower-ranked skill records introduce redundant project-specific knowledge that competes for the limited context window. Both repositories show the same overall pattern, indicating that a moderate number of retrieved skills is preferable across different repository structures.

To evaluate k_s , we varied the number of code segments rewritten during issue synthesis on both repositories. With no rewriting ($k_s = 0$), the system reduces to the baseline (62.3%). Rewriting a single segment ($k_s = 1$) increases performance to 63.2%. Performance peaks at $k_s = 5$, where each synthesized issue exposes sufficiently rich project-specific knowledge by involving multiple interacting repository entities. Increasing k_s to 7 results in a slight decrease, as larger perturbations tend to generate overly complex issues that reduce the quality of the distilled project-specific knowledge. The same trend is consistently observed across repositories, suggesting that functionality-level synthesis benefits from a moderate rewrites

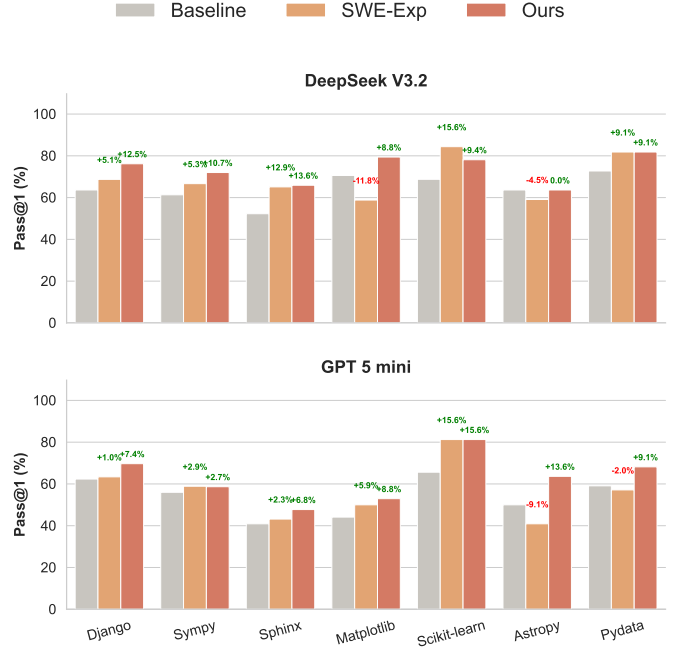


Fig. 5: Pass@1 performance comparison between Baseline and our method across seven repositories.

ing scope rather than either single-segment perturbations or overly broad rewrites.

Notably, across all settings of k_r and k_s , SkillForge consistently outperforms the baseline without project-specific knowledge, demonstrating that proactively acquired project-specific knowledge remains effective across a broad range of synthesis and retrieval configurations.

Finding 3

Moderate synthesis and retrieval scales achieve the best performance, while excessive knowledge generation or retrieval introduces redundant information that slightly degrades performance.

D. RQ4: Efficacy across Diverse Repositories

To assess whether SkillForge generalizes across different repositories, we analyze performance at the repository level. Due to space constraints, we report results for the seven repositories with the largest numbers of evaluation instances. Figure 5 compares three methods—Baseline, SWE-Exp, and SkillForge—across these seven repositories under both DeepSeek-V3.2 and GPT-5-mini.

SkillForge delivers improvements over the baseline on all seven repositories under both backbones without any regression. With DeepSeek-V3.2, the per-repo gains reach up to +13.6% (Sphinx); under GPT-5-mini, gains reach up to +15.6% (Scikit-learn). In contrast, SWE-Exp shows inconsistent behavior and suffers regressions on three repositories. Under DeepSeek-V3.2, Matplotlib drops by −11.8% and Astropy by −4.5%; under GPT-5-mini, Astropy regresses by −9.1% and Pydata by −2.0%. Because SkillForge derives

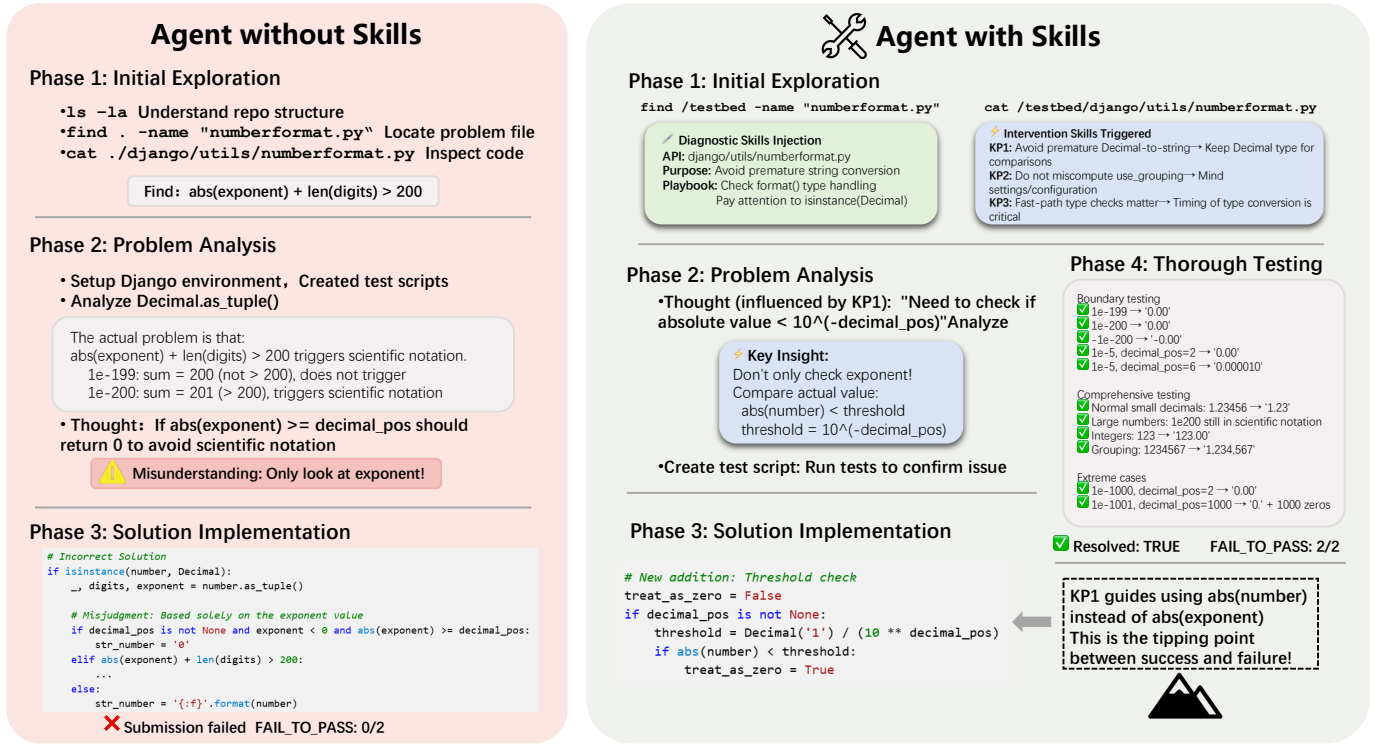


Fig. 6: Case study for *django-11206* with and without project-specific knowledge (represented as skills).

project-specific knowledge directly from each target repository through self-distillation, the resulting knowledge naturally reflects repository-specific APIs, implementation patterns, and architectural organization. Representing this knowledge as entity-grounded skills enables precise retrieval during downstream issue resolution, leading to consistently robust improvements across diverse repositories.

Finding 4

SkillForge consistently improves Pass@1 across all evaluated repositories without regression, demonstrating robust gains from proactive project-specific knowledge acquisition.

E. Case Study

To provide a more concrete view of how SkillForge changes the behavior of a repair agent, we present a case study on Django issue #11206. The bug concerns formatting extremely small `Decimal` values when a fixed number of decimal places is explicitly requested. In the buggy version, calling `nformat(Decimal("1e-200"), ".", decimal_pos=2)` returns `"1.00e-200"`, i.e., scientific notation, whereas users expect a fixed-point representation such as `"0.00"`. Correctly fixing this issue requires reasoning about when a value should be treated as numerically zero at a given precision, without breaking existing formatting behavior for other magnitudes or types.

Figure 6 contrasts two repair trajectories on this issue using the same GPT-5-mini backbone: one without project-specific

knowledge and one with SkillForge retrieving project-specific knowledge in the form of entity-grounded skills. The baseline agent locates the relevant formatting module and identifies the condition responsible for scientific notation. It then adopts a simple exponent-based heuristic to determine whether a value should be treated as zero. Although this modification fixes the observed failing example, it fails to capture the repository’s intended numeric semantics and ultimately fails all `FAIL_TO_PASS` tests (0/2).

In contrast, the skill-enhanced agent follows a longer but more structured reasoning process. After accessing the same module, the retrieved project-specific knowledge highlights two repository-specific insights: (i) preserve the existing `Decimal` formatting pipeline instead of introducing alternative representations, and (ii) reason about numerical equivalence using the repository’s precision semantics rather than a simple exponent heuristic. Guided by these insights, the agent derives a threshold-based solution that integrates naturally with the existing formatting logic. It further validates the implementation using representative boundary, precision-sensitive, and extreme-value test cases before committing the fix, ultimately passing all `FAIL_TO_PASS` tests (2/2).

This example illustrates how proactively acquired project-specific knowledge reshapes the agent’s reasoning process during issue resolution. With skill injection, the agent (i) adopts a more faithful mathematical model of the bug (*value-based thresholding* versus *exponent-based heuristics*), (ii) preserves important design constraints of the existing implementation (e.g., maintaining `Decimal`-level operations and respecting

the original formatting path), and (iii) naturally gravitates toward a test-driven workflow that exercises subtle edge cases. In contrast, the baseline trajectory quickly converges to a plausible but brittle local heuristic that fails under systematic evaluation, underscoring the value of skill-enhanced reasoning for non-trivial software bugs.

V. DISCUSSION

A. Quality of Distilled Skills and LLM-Generated Problem Statements

We manually inspected the quality of the distilled skills and synthesized problem statements. For the distilled skills, we verified that they are grounded in the corresponding repository entities, resolution trajectories, and golden patches, providing correct and actionable project-specific guidance without hallucinated APIs or misleading advice. For the synthesized problem statements, we verified that they faithfully describe the observable failures induced by functionality-level rewriting without leaking implementation details.

We also compared the issue-type distribution of synthesized issues with that of real SWE-bench issues from the same repositories. The two distributions do not overlap, suggesting that the synthesized issues do not merely reproduce the same categories of real issues used for evaluation. Instead, they tend to expose finer-grained model biases in how the current LLM uses project-specific functions, APIs, and cross-function interactions. Thus, the benefit of SkillForge is less about teaching the agent to solve similar issue types and more about surfacing project-specific function-use pitfalls that can transfer across different real issue categories.

B. Threats to Validity

a) *External validity.*: SkillForge relies on repository tests to synthesize issues and distill project-specific knowledge. Consequently, code regions that are rarely exercised by runnable tests contribute fewer learning signals, which may reduce the effectiveness of the learned knowledge in repositories with limited test coverage.

b) *Internal validity.*: Following standard SWE-bench-style synthetic instance synthesis pipelines [35]–[37], problem statements for synthesized issues are generated using an LLM. We manually verified that the synthesized failures correspond to executable bugs induced by functionality-level rewriting and that the generated descriptions correctly reflect observable behaviors without leaking implementation details. However, as in prior work, LLM-generated problem statements may still differ from developer-written issue reports.

VII. RELATED WORK

A. Project-Specific Knowledge for SWE Agents

Recent SWE agents [43]–[53] have increasingly moved beyond one-shot issue solving toward *self-evolving* behavior, where the agent improves by distilling reusable guidance from prior histories, trajectories, and execution feedback. These methods can be broadly grouped by the source of their evolution signal. One line of work evolves from repository

history. EvoCoder [24] and SWE-Exp [2] accumulate past interaction trajectories and distill abstract problem-solving patterns to guide future tasks. ExpeRepair [26] organizes previously resolved bugs into structured memories for later patch generation, while MemGovern [25] harvests community debugging traces from GitHub to improve the agent’s decision-making process. Another line of work evolves from online trajectories produced on the current instance. SWE-Debate [28] iteratively strengthens solutions through multi-agent debate, SAGE [27] abstracts high-level guidance from trial-and-error grounding during test-time scaling, and Live-SWE-agent [29] evolves its own runtime scaffold while solving the current task. Unlike these reactive paradigms, SkillForge targets the cold-start problem in project-specific issue resolution. Instead of acquiring project-specific knowledge only after accumulating repository history or online exploration trajectories, it proactively derives such knowledge directly from the repository by synthesizing project-specific issues, and represents this knowledge as entity-grounded skills for future issue resolution.

B. SWE-Style Instance Synthesis

Recent work [54]–[56] has explored synthesizing software issues or executable environments to improve software engineering agents. Existing methods primarily view synthesized instances as scalable training data. SWE-Smith [35] constructs synthetic bug-fixing tasks by independently rewriting individual API-level functions. R2E-Gym [37] generates executable SWE environments through test generation and commit back-translation to support large-scale agent training. BugPilot [36] synthesizes feature-development tasks that unintentionally introduce bugs, producing realistic development scenarios for subsequent model training.

In contrast, SkillForge uses synthetic issues for a fundamentally different purpose: proactive project-specific knowledge acquisition rather than training models. Instead of maximizing the number or diversity of synthesized tasks, SkillForge aims to expose project-specific knowledge that cannot be readily inferred from static repositories alone. Its issue construction also differs in granularity: rather than rewriting isolated API-level functions, randomly selecting several functions for joint rewriting, or generating new features, SkillForge starts from repository-tested functionalities and uses test execution traces to identify the code regions that jointly implement the same functionality. Solving the resulting issues reveals how the agent’s general coding priors diverge from project-specific implementation patterns and API interactions.

VIII. CONCLUSION

We presented SkillForge, a self-distillation framework for addressing the cold-start problem in project-specific issue resolution. Rather than relying on repository history or costly per-issue test-time exploration, SkillForge proactively derives project-specific knowledge directly from the repository by synthesizing project-specific issues and distilling their resolution trajectories. This knowledge is represented as a dual-level, entity-grounded skill repository that supports future

issue resolution. Experiments demonstrate that SkillForge consistently improves issue resolution performance, yielding absolute Pass@1 gains of +5.8%/+5.6% on SWE-bench Verified and +5.8%/+4.1% on SWE-bench Pro for DeepSeek-V3.2 and GPT-5-mini, respectively. Overall, our results suggest that proactively acquiring project-specific knowledge before real issue resolution is an effective and scalable alternative to reactive acquisition from repository history or online exploration.

REFERENCES

- [1] A. Antoniadis, A. Örwall, K. Zhang, Y. Xie, A. Goyal, and W. Wang, “SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement,” Dec. 2024.
- [2] S. Chen, S. Lin, X. Gu, Y. Shi, H. Lian, L. Yun, D. Chen, W. Sun, L. Cao, and Q. Wang, “Swe-exp: Experience-driven software issue resolution,” *arXiv preprint arXiv:2507.23361*, 2025.
- [3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” *arXiv preprint arXiv:2310.06770*, 2023.
- [4] D. Ma, S. Chen, Y. Yang, Y. Shi, Y. Yan, and X. Gu, “Llm agents can see code repositories,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.14061>
- [5] P. Gao, Z. Tian, X. Meng, X. Wang, R. Hu, Y. Xiao, Y. Liu, Z. Zhang, J. Chen, C. Gao *et al.*, “Trae agent: An llm-based agent for software engineering with test-time scaling,” *arXiv preprint arXiv:2507.23370*, 2025.
- [6] P. T. J. Kon, A. Pradeep, A. Chen, A. P. Ellis, W. Hunt, Z. Wang, J. Yang, and S. Thompson, “Swe-prot\`eg\`: Learning to selectively collaborate with an expert unlocks small language models as software engineering agents,” *arXiv preprint arXiv:2602.22124*, 2026.
- [7] M. Jiang, Y. Ruan, L. Lastras, P. Kapanipathi, and T. Hashimoto, “Putting it all into context: Simplifying agents with lclms,” *arXiv preprint arXiv:2505.08120*, 2025.
- [8] X. Jiang, G. Li, J. Qian, X. Shi, C. Li, H. Zhu, Z. Wang, J. Zhang, Z. Zhao, L. Wu *et al.*, “Koco-bench: Can large language models leverage domain knowledge in software development?” *arXiv preprint arXiv:2601.13240*, 2026.
- [9] Y. Dong, X. Jiang, J. Qian, T. Wang, K. Zhang, Z. Jin, and G. Li, “A survey on code generation with llm-based agents,” *arXiv preprint arXiv:2508.00083*, 2025.
- [10] X. Gu, H. Zhang, and S. Kim, “Deep code search,” in *Proceedings of the 40th international conference on software engineering*, 2018, pp. 933–944.
- [11] X. Gu, H. Zhang, D. Zhang, and S. Kim, “Deep api learning,” in *Proceedings of the 2016 24th ACM SIGSOFT international symposium on foundations of software engineering*, 2016, pp. 631–642.
- [12] H. Jia, E. T. Barr, and S. Mechtaev, “Compressing code context for LLM-based issue resolution,” *arXiv preprint arXiv:2603.28119*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.28119>
- [13] W. Ma, Z. Chen, J. Gu, T. Li, S. Liu, and L. Jiang, “Same signal, different semantics: A cross-framework behavioral analysis of software engineering agents,” *arXiv preprint arXiv:2605.18332*, 2026. [Online]. Available: <https://arxiv.org/abs/2605.18332>
- [14] Y. Shi, J. Xu, K. Fu, W. Zeng, S. He, L. Zhang, Y. Liu, Z. Zhao, T. Y. Zhuo, J. Cao, S. Ye, T. Liu, K. Cai, S.-C. Cheung, and X. Gu, “Swe-bench promax: Benchmarking agents on large-scale multilingual code refactoring,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.09802>
- [15] H. Lin, S. Chen, X. Gu, Y. Shi, C. Pan, J. Ge, M. Li, J. Huang, M. Chuang, B. Shen, and H. Guan, “Know before fix: Qa-driven repository knowledge acquisition for software issue resolution,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.11111>
- [16] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, Nov. 2024.
- [17] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, “Openhands: An open platform for ai software developers as generalist agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2407.16741>
- [18] G. A. Oliva, G. K. Rajbahadur, A. Bhatia, H. Zhang, Y. Chen, Z. Chen, A. Leung, D. Lin, B. Chen, and A. E. Hassan, “SPICE: An automated SWE-Bench labeling pipeline for issue clarity, test coverage, and effort estimation,” *arXiv preprint arXiv:2507.09108*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.09108>
- [19] Y. Wang, M. Pradel, and Z. Liu, “Are “solved issues” in SWE-bench really solved correctly? an empirical study,” *arXiv preprint arXiv:2503.15223*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.15223>
- [20] T. Ahmed, J. Ganhotra, A. Shinnar, and M. Hirzel, “Investigating test overfitting on SWE-bench,” *arXiv preprint arXiv:2511.16858*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.16858>
- [21] B. Yu, Y. Cao, Y. Zhang, L. Lin, J. Xu, Z. Zhong, Q. Xu, G. Wang, J. Cao, S.-C. Cheung, P. He, and L. Briand, “SWE-ABS: Adversarial benchmark strengthening exposes inflated success rates on test-based benchmark,” *arXiv preprint arXiv:2603.00520*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.00520>
- [22] B. Yu, Y. Zhu, P. He, and D. Kang, “UTBoost: Rigorous evaluation of coding agents on SWE-Bench,” *arXiv preprint arXiv:2506.09289*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.09289>
- [23] S. Garg, B. Steenhoeck, and Y. Huang, “Saving SWE-Bench: A benchmark mutation approach for realistic agent evaluation,” *arXiv preprint arXiv:2510.08996*, 2025, accepted at CAIN 2026. [Online]. Available: <https://arxiv.org/abs/2510.08996>
- [24] Y. Lin, Y. Ma, R. Cao, B. Li, F. Huang, X. Gu, and Y. Li, “Llms as continuous learners: Improving the reproduction of defective code in software issues,” *arXiv preprint arXiv:2411.13941*, 2024.
- [25] Q. Wang, Z. Cheng, S. Zhang, F. Liu, R. Xu, H. Lian, K. Wang, X. Yu, J. Yin, S. Hu *et al.*, “Memgovern: Enhancing code agents through learning from governed human experiences,” *arXiv preprint arXiv:2601.06789*, 2026.
- [26] F. Mu, J. Wang, L. Shi, S. Wang, S. Li, and Q. Wang, “Experepair: Dual-memory enhanced llm-based repository-level program repair,” *arXiv preprint arXiv:2506.10484*, 2025.
- [27] H. Hayashi, B. Pang, W. Zhao, Y. Liu, A. Gokul, S. Bansal, C. Xiong, S. Yavuz, and Y. Zhou, “Self-abstraction from grounded experience for plan-guided policy refinement,” *arXiv preprint arXiv:2511.05931*, 2025.
- [28] H. Li, Y. Shi, S. Lin, X. Gu, H. Lian, X. Wang, Y. Jia, T. Huang, and Q. Wang, “Swe-debate: Competitive multi-agent debate for software issue resolution,” *arXiv preprint arXiv:2507.23348*, 2025.
- [29] C. S. Xia, Z. Wang, Y. Yang, Y. Wei, and L. Zhang, “Live-swe-agent: Can software engineering agents self-evolve on the fly?” *arXiv preprint arXiv:2511.13646*, 2025.
- [30] Z. Fan, K. Vasilevski, D. Lin, B. Chen, Y. Chen, Z. Zhong, J. M. Zhang, P. He, and A. E. Hassan, “SWE-Effi: Re-evaluating software AI agent system effectiveness under resource constraints,” *arXiv preprint arXiv:2509.09853*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.09853>
- [31] OpenAI, “Swe-bench verified,” <https://openai.com/index/introducing-swe-bench-verified/>, 2024.
- [32] X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane *et al.*, “Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?” *arXiv preprint arXiv:2509.16941*, 2025.
- [33] P. Chang, Y. Fang, S. Chen, Y. Shi, B. Shen, and X. Gu, “Test vs mutant: Adversarial llm agents for robust unit test generation,” *arXiv preprint arXiv:2602.08146*, 2026.
- [34] Y. Chen, T. Ahmed, R. Jabbarvand, and M. Hirzel, “Can old tests do new tricks for resolving SWE issues?” *arXiv preprint arXiv:2510.18270*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.18270>
- [35] J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang, “Swe-smith: Scaling data for software engineering agents,” *arXiv preprint arXiv:2504.21798*, 2025.
- [36] A. Sonwane, I. White, H. Lee, M. Pereira, L. Caccia, M. Kim, Z. Shi, C. Singh, A. Sordani, M.-A. Côté, and X. Yuan, “Bugpilot: Complex bug generation for efficient learning of swe skills,” *arXiv preprint arXiv:2510.19898*, 2025.
- [37] N. Jain, J. Singh, M. Shetty, L. Zheng, K. Sen, and I. Stoica, “R2e-gym: Procedural environments and hybrid verifiers for scaling open-weights swe agents,” *arXiv preprint arXiv:2504.07164*, 2025.
- [38] SWE-agent, “mini-swe-agent: The minimal ai software engineering agent,” GitHub repository, SWE-agent, 2025, <https://github.com/SWE-agent/mini-swe-agent>.

- [39] Y. Guo, Y. Xiao, J. M. Zhang, M. Harman, Y. Lou, Y. Liu, and Z. Chen, "Eet: Experience-driven early termination for cost-efficient software engineering agents," *arXiv preprint arXiv:2601.05777*, 2026.
- [40] Y. Wang, Y. Shi, M. Yang, R. Zhang, S. He, H. Lian, Y. Chen, S. Ye, K. Cai, and X. Gu, "Swe-pruner: Self-adaptive context pruning for coding agents," *arXiv preprint arXiv:2601.16746*, 2026.
- [41] A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong *et al.*, "Deepseek-v3. 2: Pushing the frontier of open large language models," *arXiv preprint arXiv:2512.02556*, 2025.
- [42] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, A. McLaughlin, A. Low, A. Ostrow, A. Ananthram *et al.*, "Openai gpt-5 system card," *arXiv preprint arXiv:2601.03267*, 2025.
- [43] Z. Pan, C. Li, W. Zhong, Y. Feng, B. Luo, and V. Ng, "Reporepair: Leveraging code documentation for repository-level automated program repair," *arXiv preprint arXiv:2603.01048*, 2026.
- [44] C. M. Team, Y. Ye, J. Tan, T. Jiang, R. Ye, Q. He, J. Yang, J. Dong, S. Liang, C. Yue *et al.*, "Yet even less is even better for agentic, reasoning, and coding llms," *arXiv preprint arXiv:2604.00824*, 2026.
- [45] M. Raghavendra, A. Gunjal, B. Liu, and Y. He, "Agentic rubrics as contextual verifiers for swe agents," *arXiv preprint arXiv:2601.04171*, 2026.
- [46] G. Chen, F. Meng, J. Zhao, M. Li, D. Cheng, H. Song, J. Chen, Y. Lin, H. Chen, X. Zhao *et al.*, "Beyondswe: Can current code agent survive beyond single-repo bug fixing?" *arXiv preprint arXiv:2603.03194*, 2026.
- [47] J. Xu, K. Deng, W. Li, S. Yu, H. Tang, H. Huang, Z. Lai, Z. Zhan, Y. Wu, C. Zhang *et al.*, "Swe-compass: Towards unified evaluation of agentic coding abilities for large language models," *arXiv preprint arXiv:2511.05459*, 2025.
- [48] W. Zeng, Y. Wang, C. Hu, Y. Shi, C. Wan, H. Zhang, and X. Gu, "Pruning the unsurprising: Efficient code reasoning via first-token surprisal," *arXiv preprint arXiv:2508.05988*, 2025.
- [49] W. Zeng, Y. Shi, X. Gu, C. Hu, C. Wang, Y. Cui, H. Zhou, M. Qi, J. Wangni, Z. Yu *et al.*, "Dockerless: Environment-free program verifier for coding agents," *arXiv preprint arXiv:2606.28436*, 2026.
- [50] W. Zeng, X. Zhang, Y. Shi, C. Hu, Y. Chen, B. Shen, and X. Gu, "Glimprouter: Efficient collaborative inference by glimpsing one token of thoughts," *arXiv preprint arXiv:2601.05110*, 2026.
- [51] C. Hu, W. Zeng, Y. Shi, B. Shen, and X. Gu, "In line with context: Repository-level code generation via context inlining," *arXiv preprint arXiv:2601.00376*, 2026.
- [52] S. Gao, W. Zeng, Z. Yu, J. Wangni, C. Wang, K. Cai, S. He, and M. R. Lyu, "Swe-mem: Learning adaptive memory management for long-horizon coding agents," *arXiv preprint arXiv:2606.28434*, 2026.
- [53] J. Huang, S. Yun, S. Chen, X. Gu, and B. Shen, "Planning over actions: Agentic reasoning for semi-structured table question answering," *Information Processing & Management*, vol. 64, no. 1, p. 105092, 2027.
- [54] G. Yu, G. Tan, H. Huang, Z. Zhang, P. Chen, R. Natella, Z. Zheng, and M. R. Lyu, "A survey on failure analysis and fault injection in ai systems," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 1, pp. 1–42, 2026.
- [55] R. Hu, C. Peng, X. Wang, J. Xu, and C. Gao, "Repo2Run: Automated building executable environment for code repository at scale," *arXiv preprint arXiv:2502.13681*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.13681>
- [56] L. Wang, L. Ramalho, A. Celestino, P. A. Pham, Y. Liu, U. K. Sinha, A. Portillo, O. Osunwa, and G. Maduekwe, "SWE-Bench++: A framework for the scalable generation of software engineering benchmarks from open-source repositories," *arXiv preprint arXiv:2512.17419*, 2025. [Online]. Available: <https://arxiv.org/abs/2512.17419>